

O sa ma iau sa descriu functionalitatea temei incepand de la main, intrand prin toate ramificatiile acesteia in alte functii si modul in care functioneaza codul si cum am facut implementarea.

In main totul incepe prin initializarea programului, Init familiarizeaza programul nostru cu faptul ca este rulat pe mai multe procese, ii comunicam care e rank-ul procesului curent si cate procese avem in total in cadrul programului nostru.

Dupa ce am comunicat programului datele de mai sus, citim din fisierul cu acelasi rang pe care il are si procesul curent si alocam memoria in functie de nr de lookups citit.

Folosim MPIAllgather ca fiecare proces sa afle id-ul celorlalte procese. Practic procesele fac schimb intre ele de ID-uri. Construim pentru fiecare proces map-ul de id-uri, global ring-ul si finger table-ul (cu ajutorul functiei de la TODO1).

Pentru TODO1:

functia build_finger_table trece prin cele 4 noduri responsabile si seteaza valoarea nodului curent si al celui succesor

find_succesor_simple trece prin toate nodurile, in ordine folosind sorted_ids, cand a gasit un indice egal sau mai mare cu cheia careia vrem sa ii asignam un nod responsabil, returnam valoarea, daca nu gasim, e clar ca se afala in intervalul de la ultimul si primul indice deci returnam indicele de pe prima pozitie ca nod raspunzator/succesor

Dupa care vom astepta pana cand toate procesele noastre pe care le avem initializate ajung in punctul acesta.

Pentru TODO-ul 4:

Construim mesajul de lookup pentru prima cheie din lista. Mesajul il trimitem procesului curent, pe care il vom decoda in todo 5. La pasul urmator din main, am setat campurile conform comentariilor din tema.

Pentru TODO 5:

Aici prelucram toate mesajele primite, in functie de tag, cate request-uri avem, si cate procese au terminat executia.

Daca numarul de lookup-uri este 0, vom trimite la toate celelalte procese mesajul done, pentru a le instiinta ca unul dintre procese tocmai s-a terminat.

Fiecare proces in parte va astepta sa primeasca done de la celelalte procese, am luat un contor, done_received care contorizeaza cate mesaje done au fost primite de fiecare proces in parte.

Cand ajungem la un numar de mesaje done egal cu world_size - 1, inseamna ca toate procesele si-au terminat executia si singurul lucru care ne ramane de facut este sa asteptam pana cand se epuizeaza toate lookup-urile ramase, asta daca nu cumva s-au epuizat deja.

Se asteapta dupa mesaje si le procesam in functie de tag-ul lor,

Daca avem REQUEST, vom folosi functia de la TODO3, pentru request-uri

functia handle_lookup_request:

Functia primeste o refernta la un mesaj, pe care il actualizam cu self.id.

Daca cheia mesajului este chiar id-ul curent, nodul responsabil va fi chiar cel curent altfel, verificam daca cheia se afla in interval.

Daca se afla, nodul responsabil devine succesorul nodului curent

Daca avem un nod responsabil, actualizam path-ul din mesaj si adaugam nodul responsabil, incrementam lungimea path-ului si trimitem procesului care a initiat request-ul, nodul responsabil in reply.

Altfel, inseamna ca trebuie sa continuam cautarea pentru nodul responsabil si vom lua urmatorul nod folosind functia `closest_preceding_function` de la TODO 2

Pentru TODO2:

Parcurgem intrarile din finger table de la cel mai mare la cel mai mic, daca avem finger-ul in interval, inseamna ca am gasit nodul cautat

Daca in urma cautarilor se constata ca niciun finger nu este cel dorit, intoarcem succesorul

Daca avem REPLY, afisam calea lookup-ului si cresc index-ul curent, daca nu am ajuns la ultimul index trimit lookup-ul urmator.

E posibil sa nu mai am lookup-uri deci daca `idx_curent` ajunge la final, am terminat si acest proces si putem trimite `tag_done` la procesele ramase

Daca avem tag DONE incrementam numarul de procese care si-au terminat treaba

Apelam `MPI_Finalize`, pentru a marca faptul ca iesim din zona de procesare in paralel si programul isi termina executia.